LABORATORIO 2: RECURSIÓN Y FUNCIONES DE ORDEN SUPERIOR EN HASKELL

Gabriel Andrés Camargo Barón

1. PUNTO 1 - RECURSIÓN Y FUNCIONES DE ORDEN SUPERIOR

a) Definición de funciones usando Recursión Explícita (Punto1_Recursion.hs):

```
module Punto1_Recursion where

tamano :: [a] → Int
tamano []     = 0
tamano (_:xs) = 1 + tamano xs

ultimo :: [a] → a
ultimo []     = error "La lista se encuentra vacia"
ultimo [x]    = x
ultimo (_:xs) = ultimo xs

duplicar :: [Int] → [Int]
duplicar []     = []
duplicar (x:xs) = (x * 2) : duplicar xs

eliminarPares :: [Int] → [Int]
eliminarPares [] = []
eliminarPares (x:xs)
| x mod 2 /= 0 = x : eliminarPares xs
| otherwise      = eliminarPares xs
```

Pruebas ejecutadas en consola para capturas de pantalla:

- tamano [1, 3, 5, 7, 9, 11] → 6
- ultimo [15, 30, 45, 60, 75] → 75
- duplicar [3, 6, 9, 12] → [6, 12, 18, 24]
- eliminarPares [2, 5, 8, 11, 14, 17] → [5, 11, 17]

b) Equivalencia usando Funciones de Orden Superior (Punto1_OrdenSuperior.hs):

```
module Punto1_OrdenSuperior where

duplicarOS :: [Int] → [Int]
duplicarOS = map (∗2)

eliminarParesOS :: [Int] → [Int]
eliminarParesOS = filter odd
```

Pruebas ejecutadas en consola

- duplicarOS [5, 10, 15, 20] → [10, 20, 30, 40]
- eliminarParesOS [1, 2, 3, 4, 5, 6, 7, 8] → [1, 3, 5, 7]

2. PUNTO 2 - COMPRENSIÓN DE LISTAS

Código Haskell (Punto2_Comprension.hs):

```
module Punto2_Comprension where

cuadradosDiez :: [Int]
cuadradosDiez = [x^2 | x ← [1..10]]

divisiblesTres :: [Int]
divisiblesTres = [x | x ← [1..30], x mod 3 == 0]

paresOrdenados :: [(Int, Int)]
paresOrdenados = [(x, y) | x ← [1..5], y ← [1..5], x < y]

divisores :: Int → [Int]
divisores n = [x | x ← [1..n], n `mod` x == 0]

divisoresPropios :: Int → [Int]
divisoresPropios n = [x | x ← [1..n-1], n mod x == 0]

esPerfecto :: Int → Bool
esPerfecto n = sum (divisoresPropios n) == n
```

Pruebas ejecutadas en consola

- cuadradosDiez → [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
- divisiblesTres → [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
- paresOrdenados → [(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]
- divisores 24 → [1, 2, 3, 4, 6, 8, 12, 24]
- divisoresPropios 28 → [1, 2, 4, 7, 14]
- esPerfecto 28 → True
- esPerfecto 18 → False

3. PUNTO 3 - TRANSFORMACIÓN Y FILTRADO AVANZADO

Código Haskell (Punto3_Combinadas.hs):

```
module Punto3_Combinadas where

import Data.Char (toUpper)

mayoresQue10 :: [Int] → Int
mayoresQue10 xs = length (filter (>10) xs)

productoListaOS :: [Int] → Int
productoListaOS = product

celsiusAFahrenheit :: [Double] → [Double]
celsiusAFahrenheit = map (\c → (c * 9/5) + 32)

ajustarImpares :: [Int] → [Int]
ajustarImpares = map (\x → if odd x then x + 1 else x)

nombresLargos :: [String] → [String]
nombresLargos = filter (\nom → length nom > 5)

soloPositivos :: [Int] → [Int]
soloPositivos = filter (>0)
```

```
edadesUniversitarias :: [Int] -> [Int]
edadesUniversitarias = filter (\e -> e >= 18 && e <= 25)

preciosConImpuesto :: [Double] -> [Double]
preciosConImpuesto precios = map (* 1.19) (filter (> 1000) precios)

longitudVocales :: [String] -> [Int]
longitudVocales palabras = map length (filter empiezaConVocal palabras)
where
empiezaConVocal p = not (null p) && head (map toUpper p) elem "AEIOU"

contadorPares :: [Int] -> Int
contadorPares xs = length (filter even xs)

todosPares :: [Int] -> Bool
todosPares = all even

nombresCorrectos :: [String] -> Bool
nombresCorrectos = all (\n -> not (null n) && toUpper (head n) elem ['A'..'Z'])

menoresQue100 :: [Int] -> [Int]
menoresQue100 = takeWhile (<100)

sinCerosIniciales :: [Int] -> [Int]
sinCerosIniciales = dropWhile (== 0)
```

Pruebas ejecutadas en consola

- mayoresQue10 [4, 15, 2, 28, 9, 33] → 3
- celsiusAFahrenheit [0.0, 20.0, 37.0] → [32.0, 68.0, 98.6]
- nombresLargos ["Gabriel", "Ana", "Fernando", "Luis", "Valentina"] → ["Gabriel", "Fernando", "Valentina"]
- soloPositivos [-8, 12, -3, 0, 19, -1] → [12, 19]
- preciosConImpuesto [800.0, 1200.0, 500.0, 2500.0] → [1428.0, 2975.0]
- contadorPares [1, 4, 7, 10, 13, 16, 19] → 3
- todosPares [2, 4, 8, 12] → True
- sinCerosIniciales [0, 0, 0, 4, 8, 0, 2] → [4, 8, 0, 2]

4. PUNTO 4 - CURRYING Y FUNCIONES COMO ARGUMENTOS Y RETORNO
Código Haskell (Punto4_Currying.hs):

```
module Punto4_Currying where

procesarLecturas :: (a → b) → [a] → [b]
procesarLecturas f lista = map f lista

crearFiltro :: (a → Bool) → [a] → [a]
crearFiltro criterio = filter criterio

enRango :: Ord a ⇒ a → a → (a → Bool)
enRango minimo maximo = \x → x >= minimo && x <= maximo

potenciador :: Int → (Int → Int)
potenciador n = \base → base ^ n
```

Pruebas ejecutadas en consola

- procesarLecturas (\c → (c * 9/5) + 32) [10.0, 25.0, 30.0] → [50.0, 77.0, 86.0]
- let filtroMayor15 = crearFiltro (> 15) in filtroMayor15 [5, 12, 18, 3, 22, 9] → [18, 22]
- let rangoUniversitario = enRango 18 25 in rangoUniversitario 21 → True
- let cubo = potenciador 3 in cubo 4 → 64

5. RETROSPECTIVA

1. Tiempo total invertido: Dediqué un tiempo estimado de 5 a 6 horas hombre repartidas entre el análisis conceptual de funciones de orden superior, la codificación de cada módulo y la ejecución de pruebas en el entorno virtual.
2. Estado actual: Completado al 100%. Todas las funciones solicitadas en los 4 puntos se encuentran implementadas, modularizadas y validadas mediante ejemplos prácticos sin errores de compilación.
3. Mayor logro: Lograr desacoplar la recursión explícita sustituyéndola por combinaciones elegantes de map, filter, takeWhile y lambdas, además de

comprender el concepto de Currying para construir generadores de funciones personalizadas.

4. Mayor problema técnico: Ajustar correctamente las condiciones de bordes en expresiones lambda con guardas y la compatibilidad de tipos numéricos (Int frente a Double o Fractional). Se solucionó realizando comprobaciones inmediatas en GHCi/Binder y afinando las firmas de tipo explícitas.
5. Trabajo en equipo y mejora: Trabajo individual riguroso para mantener el código claro y debidamente documentado. Me comprometo a solicitar retroalimentación previa en los horarios de atención docente para optimizar aún más las expresiones idiomáticas en Haskell.
6. Referencias:

- Lipovača, M. (2011). Learn You a Haskell for Great Good!: A Beginner's Guide. No Starch Press.
- Alonso, J. A. (2023). Funciones básicas en Haskell e introducción a la programación funcional. Universidad de Sevilla. https://www.cs.us.es/~jalonso/cursos/i1m/doc/Funciones_basicas.html